

Module 4 — The models

Revolut's topic 4, verbatim: "**Basic ML models, pros and cons of different models.**" Every model here is Module 3's machinery in a different shape: a function with parameters, trained by minimising a loss, sitting somewhere on the bias–variance curve. For each one: the idea in two lines, how it learns, pros, cons, when you'd use it at Reolut — and a 30-second answer.

HOW TO ANSWER "PROS AND CONS OF X"

Five beats, always the same: what it does → how it learns → two strengths → two weaknesses → when I'd use it. Thirty seconds. Then stop.

And one honest sentence you can use for any model you haven't worked with: "I haven't used that in production; what I know is what it's for and where it sits against the alternatives." For a breadth interview that is a good answer.

D0 The map

FAMILY	MODELS	CORE IDEA
Linear	linear regression · logistic regression (· SVM with a linear kernel)	a weighted sum of the features
Instance-based	k-nearest neighbours	"similar cases have similar outcomes"
Probabilistic	naive Bayes	Bayes' theorem with an independence shortcut
Trees	decision tree	a flowchart of if/else questions
Ensembles of trees	random forest (bagging) · gradient boosting (XGBoost, LightGBM, CatBoost)	many trees are better than one
Neural networks	feed-forward · CNN (images) · RNN / transformer (sequences, text)	layers of weighted sums with non-linearities; learns its own features
Unsupervised	k-means · PCA · isolation forest	structure without labels

The practical truth worth saying out loud: on tabular business data — customers, transactions, credit — **gradient-boosted trees** are usually the strongest off-the-shelf model, **logistic regression** is the interpretable baseline and the regulated-industry standard, and **neural networks** win on images, text and sequences, or when you have enormous data and can learn representations (which is what PRAGMA does with event sequences).

D1 Linear regression

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n \text{ trained by minimising MSE}$$

What it does: predicts a number as a weighted sum of the features. **How it learns:** finds the weights that minimise the squared error — in closed form (the normal equation) or by gradient descent.

Reading it: w_1 is the change in the prediction when x_1 rises by one unit, *holding the other features fixed*. w_0 (the intercept, or bias term) is the prediction when all features are zero. A bigger weight does *not* mean a more important feature unless the features are on the same scale.

Assumptions (a classic question)

- **Linearity** — the relationship really is roughly a straight line (or you've transformed features so it is).
- **Independent errors** — one case's error tells you nothing about another's (violated in time series).
- **Constant error variance** (homoscedasticity) — errors are as large for big customers as for small ones.
- **Normally distributed errors** — needed for confidence intervals and p-values on the weights, not for prediction itself.
- **No strong multicollinearity** — features aren't near-copies of each other.

Multicollinearity: if $z = x + y$ exactly, the weights are not uniquely defined — infinitely many combinations give the same predictions, and the software either fails or returns arbitrary huge weights. If $z = x + y + \text{a little noise}$, it runs, but the weights are unstable: tiny changes in the data swing them wildly. Predictions can still be fine; interpretation is destroyed. Fix: drop one, or regularise (ridge).

PROS

Simple, fast, needs little data · fully interpretable (one weight per feature) · a strong baseline · low variance

CONS

Only captures linear relationships unless you engineer features · sensitive to outliers (squared loss) · unstable with correlated features · high bias on complex patterns

Use it for: forecasting a quantity with a few well-understood drivers; as the baseline before anything fancier. For prices, amounts and other skewed targets: predict $\log(y)$.

30 SECONDS — "LINEAR REGRESSION: PROS AND CONS"

"It predicts a number as a weighted sum of the features, with weights chosen to minimise squared error. Its strengths are simplicity and interpretability — each weight is the effect of one feature holding the others fixed — and it needs little data and is hard to overfit. Its weaknesses: it only captures linear effects unless I engineer the features, squared error makes it sensitive to outliers, and correlated features make the weights unstable. I'd use it as a baseline and when explaining the drivers matters more than the last point of accuracy."

D2 Logistic regression

$$z = w_0 + w_1x_1 + \dots + w_nx_n \quad p = \sigma(z) = 1 / (1 + e^{-z}) \quad \text{trained by minimising log loss}$$

What it does: classification. It computes the same weighted sum as linear regression, then squashes it through the **sigmoid** into a probability between 0 and 1. **How it learns:** gradient descent on the log loss (= maximum likelihood).

- **Sigmoid:** an S-shaped curve; $\sigma(0) = 0.5$; large positive $z \rightarrow \text{near } 1$; large negative $\rightarrow \text{near } 0$.
- **Is it a linear model?** Yes. The decision boundary (where $p = 0.5$, i.e. $z = 0$) is a straight line / flat plane, and the model is linear in the **log-odds**: $\log(p / (1 - p)) = z$.
- **Reading it:** raising x_1 by one unit adds w_1 to the log-odds, i.e. multiplies the odds by $e^{(w_1)}$. "One more month late multiplies the odds of default by 2.3."
- Despite the name, it's a **classification** model.

PROS

Interpretable — weights are odds ratios · outputs well-calibrated probabilities · fast, scales to millions of rows and sparse features · stable, low variance · easy to regularise (L1/L2) · accepted by regulators

CONS

Linear boundary: misses interactions and non-linear effects unless you build them in (bins, products) · needs scaled features when regularised · sensitive to outliers and to highly correlated features · usually beaten by boosted trees on raw tabular data

Use it for: the baseline for any classification problem; **credit scorecards** (the industry standard for decades precisely because every point can be explained to a customer and a regulator); high-dimensional sparse

data like text.

30 SECONDS – "WHAT IS LOGISTIC REGRESSION? IS IT LINEAR?"

"It's a classifier: a weighted sum of the features passed through a sigmoid to give a probability, trained by minimising log loss. It is a linear model — linear in the log-odds, so the decision boundary is a straight line. Pros: interpretable, each weight is an odds ratio; well-calibrated probabilities; fast and stable; and regulators accept it, which is why credit scorecards use it. Cons: it can't capture non-linear effects or interactions unless I engineer them, so on raw tabular data boosted trees usually beat it. I always fit it first as the baseline."

EXERCISES D1-D2

1. A linear regression for monthly spend has weights: income (in €k) = 12, age (in years) = -0.5. Which feature is "more important"? What would you need to do to compare them?
2. Your dataset has balance_eur, balance_usd and their sum. What happens to a linear model, and what do you do?
3. A logistic regression for default has weight 0.7 on "months late". What does one extra month do to the odds? If $z = 0$ for a customer, what's the predicted probability?
4. Default depends on the *combination* "young AND high credit utilisation" but on neither alone. Will plain logistic regression find it? How could you help it?
5. Why do banks still use logistic regression for credit decisions when boosted trees score higher?

D3 k-nearest neighbours (k-NN)

What it does: to predict for a new case, find the k most similar cases in the training data and take a vote (classification) or an average (regression). **How it learns:** it doesn't — it stores the data ("lazy learning"); all the work happens at prediction time.

- **k** is the bias–variance dial: $k = 1$ follows every point (high variance, overfits); large k averages over big neighbourhoods (high bias). Choose by cross-validation.
- **Similarity = distance**, so features **must be scaled** — otherwise the one in the largest units dominates.
- **Curse of dimensionality:** with many features, every point is far from every other, "nearest" stops meaning anything, and you'd need exponentially more data.

PROS

Dead simple, no training · naturally non-linear · easy to explain ("customers like you...") · new data can be added instantly

CONS

Slow and memory-hungry at prediction time on large data · breaks down in high dimensions · needs scaling and a meaningful distance · sensitive to irrelevant features

Use it for: small, low-dimensional problems; baselines; and, in modern form, **nearest-neighbour search on embeddings** — "find customers similar to this one" in PRAGMA's embedding space, or the retrieval step in RAG.

D4 Naive Bayes

What it does: applies Bayes' theorem (Module 1) to get $P(\text{class} \mid \text{features})$, with one simplifying — "naive" — assumption: the features are **independent given the class**, so their likelihoods just multiply. **How it learns:** by counting — how often each feature value appears in each class.

$$P(\text{spam} \mid \text{words}) \propto P(\text{spam}) \times P(\text{word}_1 \mid \text{spam}) \times P(\text{word}_2 \mid \text{spam}) \times \dots$$

PROS

Extremely fast to train and predict · works with little data · handles thousands of features (words) · a strong text baseline

CONS

The independence assumption is almost always false → probabilities are poorly calibrated (too extreme), though the ranking is often fine · can't learn interactions

Use it for: text classification baselines (spam, routing support tickets by topic), when you need something working in an hour.

30 SECONDS – "K-NN AND NAIVE BAYES"

"k-NN predicts from the k most similar training cases — a vote or an average. No training, naturally non-linear, but slow at prediction time, it needs scaled features, and it degrades in high dimensions because distances stop being meaningful; k trades variance for bias. Naive Bayes applies Bayes' theorem assuming the features are independent given the class, so it just multiplies likelihoods learned by counting. It's very fast and a good text baseline, but the independence assumption is wrong in practice, so its probabilities are badly calibrated even when its ranking is useful."

D5 Decision trees

What it does: a flowchart of yes/no questions on the features ("months late ≥ 2 ?" → "limit < €1,000?" → ...); each leaf gives a prediction — the majority class or the average of the training cases that land there. **How it learns:** greedily. At each node it tries every feature and every threshold and picks the split that makes the two children as **pure** as possible, then recurses.

- **Purity measures:** **Gini impurity** $1 - \sum p_k^2$ or **entropy** $-\sum p_k \log p_k$ for classification (both are 0 for a pure node, maximal at 50/50); variance / MSE for regression. The improvement from a split is the **information gain**.
- **Main hyperparameters:** maximum depth, minimum samples per leaf, minimum gain to split. They control complexity: a fully grown tree memorises the training data.
- **Pruning:** cutting back branches that don't improve validation error.

PROS

Interpretable — you can read the rules · captures non-linear effects and interactions automatically · no scaling needed; handles mixed numeric / categorical data and outliers · fast

CONS

High variance: a small change in the data can produce a completely different tree · overfits easily if deep · greedy, so not globally optimal · step-shaped predictions; can't extrapolate beyond the training range · usually less accurate alone than ensembles

Use it for: when you need explicit, auditable rules; exploring which features and thresholds matter; and as the building block of the two ensembles below.

30 SECONDS – "DECISION TREES: PROS AND CONS"

"A decision tree splits the data with a sequence of yes/no questions, choosing at each step the feature and threshold that best separate the classes — measured by Gini or entropy — until the leaves are fairly pure. Pros: readable rules, non-linear effects and interactions for free, no scaling, robust to outliers. Cons: a single tree is unstable — high variance, it overfits if I let it grow — and it's greedy and gives step-like predictions. So in practice I'd use a shallow tree when I need explainable rules, and otherwise an ensemble of trees."

D6 Ensembles of trees — random forest and gradient boosting

One tree is unstable. Two ways to fix that — and the difference between them is the most likely "compare two models" question.

Random forest — many independent trees, averaged (bagging)

- Train hundreds of deep trees, each on a **bootstrap sample** (rows drawn with replacement) and with a **random subset of features** considered at each split. Average their predictions (or take a vote).
- **Why it works:** each tree has low bias and high variance; averaging many *decorrelated* trees cancels the variance. The randomness in rows and features is what decorrelates them — if all trees saw the same data they'd make the same mistakes and averaging wouldn't help.
- **Hyperparameters:** number of trees (more is better until it plateaus; it doesn't overfit by adding trees), max features per split, tree depth / min leaf size.
- Trees are independent → trivially **parallel**. Free validation via **out-of-bag** samples (each tree is tested on the rows it didn't see).

Gradient boosting — trees in sequence, each fixing the last one's errors

- Start with a simple prediction (the mean). Fit a **small** tree to the **errors** (residuals — formally, the gradient of the loss). Add a fraction of it to the model (the **learning rate**). Repeat hundreds of times.
- **Why it works:** each shallow tree is a weak learner with high bias; adding them one by one drives the **bias** down step by step.
- **Hyperparameters:** number of trees, learning rate (smaller rate → more trees needed, usually better), tree depth (3–8), subsampling of rows/columns, regularisation. Choose the number of trees by **early stopping** on validation data — unlike a forest, boosting *does* overfit if you keep adding trees.
- **XGBoost, LightGBM, CatBoost** are fast, regularised implementations. They handle missing values natively.

	RANDOM FOREST	GRADIENT BOOSTING
Trees built	independently, in parallel	sequentially, each on the previous errors
Each tree	deep (low bias, high variance)	shallow (high bias, low variance)
Mainly reduces	variance	bias
More trees	never hurts	eventually overfits → early stopping
Tuning	works well out of the box	needs care (learning rate, depth, number of trees)
Accuracy on tabular data	very good	usually the best
Robust to noisy labels / outliers	more	less (it chases errors)

Both: no scaling needed, non-linear, interactions for free, strong on tabular data. **Both lose:** the readable single tree — you get **feature importance** instead (by total gain, by **permutation** — shuffle one feature and see how much the score drops — or **SHAP** values, which explain an individual prediction). Importance is not causation, and correlated features share or steal each other's importance.

30 SECONDS — "RANDOM FOREST VS. GRADIENT BOOSTING"

"Both are ensembles of decision trees. A random forest trains many deep trees independently on bootstrap samples with random feature subsets and averages them — averaging decorrelated trees reduces variance; it parallelises, is robust, and works almost without tuning. Gradient boosting builds shallow trees sequentially, each one fitted to the errors of the model so far, which reduces bias; it's usually the most accurate on tabular data but needs careful tuning — learning rate, depth, and early stopping, because it will overfit if I keep adding trees. I'd start with a forest for a robust baseline and move to boosting — XGBoost or LightGBM — for the last points of accuracy."

EXERCISES D3-D6

1. In k-NN you go from $k = 1$ to $k = 200$ on 1,000 training points. What happens to bias and variance? What does the model do at $k = 1,000$?
2. k-NN on customers using income (in €) and age (in years), unscaled. Which feature decides who the neighbours are, and why?
3. Why is naive Bayes "naive"? Give an example of two features in a fraud problem that clearly violate the assumption.
4. A node has 80 non-defaulters and 20 defaulters. Gini impurity? After a split: left 60/0, right 20/20. Is that a good split? (Compute the weighted Gini.)
5. You retrain a deep tree each month and the rules change completely. Which ensemble addresses exactly this, and how?
6. In a random forest, why choose a random subset of features at each split instead of always the best of all features?
7. Your boosting model: training AUC keeps rising to 0.99; validation AUC peaked at round 300 and is now falling at round 1,000. What is happening and what do you do?
8. "Feature X has the highest importance, so increasing X will reduce default." What's wrong with that sentence?

D7 Support vector machines (SVM) — one paragraph

What it does: finds the boundary that separates two classes with the **widest possible margin**; only the points nearest the boundary — the **support vectors** — matter. With the **kernel trick** it can draw curved boundaries by implicitly working in a higher-dimensional space. **Pros:** effective in high dimensions and on small-to-medium data; robust. **Cons:** slow on large datasets; needs scaling and careful choice of kernel and parameters; no natural probabilities; hard to interpret. Largely displaced by boosted trees (tabular) and neural networks (everything else) — know what it is, don't dwell on it.

D8 Neural networks

What it does: stacks layers of "weighted sum $\rightarrow$ non-linear **activation**". The first layers learn simple features, later layers combine them into complex ones — the network **learns its own features** from raw data instead of relying on hand-crafted ones. **How it learns: backpropagation** computes how much each weight contributed to the error (the chain rule, layer by layer, backwards), and (stochastic) gradient descent updates all the weights.

- **Why activations:** without a non-linearity, any stack of layers collapses into one linear model. **ReLU** ($\max(0, z)$) is the default — cheap and it doesn't "saturate" like the sigmoid, whose flat ends make gradients vanish in deep networks. Sigmoid / softmax are used at the output to produce probabilities.
- **Regularisation:** dropout (randomly switch off units during training), weight decay (L2), early stopping, data augmentation.
- **Architectures, one line each:** **CNN** — shared filters that slide over an image (ID documents, cheques). **RNN / LSTM** — process a sequence step by step (older approach to text, time series). **Transformer** — **attention** lets every element of a sequence look at every other; the architecture behind LLMs and behind **PRAGMA**, which treats a customer's banking events like words in a sentence.
- **Embeddings:** learned vectors that represent things (words, customers, merchants) so that similar ones are close. The output of PRAGMA is a customer embedding; a small model on top of it does the actual task.
- **Transfer learning / fine-tuning:** take a model pre-trained on huge data and adapt it to your task with little data. This is the foundation-model idea.

PROS

State of the art on images, text, audio, sequences
· learns features automatically · scales with data
— keeps improving where other models plateau ·
flexible: any input, any output · transfer learning

CONS

Data- and compute-hungry · many hyperparameters; harder to
train and debug · a black box — hard to explain to a regulator ·
on small / medium **tabular** data, boosted trees are usually as
good or better with far less effort · latency and serving cost

30 SECONDS — "NEURAL NETWORKS: WHEN WOULD YOU USE ONE?"

"A neural network is layers of weighted sums with non-linear activations, trained by backpropagation and gradient descent; its distinguishing feature is that it learns its own representations from raw data. I'd use one for unstructured data — images, text, event sequences — or when there's enough data that learned features beat hand-made ones; that's the logic of a foundation model like PRAGMA. The costs are data and compute, tuning effort, and explainability. For an ordinary tabular problem with a few hundred thousand rows I'd reach for gradient boosting first."

D9 Unsupervised — k-means, PCA, anomaly detection

k-means clustering

What it does: groups points into k clusters. **How:** (1) place k centres at random; (2) assign every point to its nearest centre; (3) move each centre to the mean of its points; repeat 2–3 until nothing changes.

- **Choosing k:** the **elbow** method (plot within-cluster distance against k; look for the bend), the **silhouette** score, or — honestly — what's useful to the business.
- **Pros:** simple, fast, scales well. **Cons:** you must choose k; assumes roughly round, similar-sized clusters; sensitive to scaling, outliers and the random start (run it several times); there's no "right answer" to validate against.
- **Alternatives, one line:** **DBSCAN** — clusters by density, finds arbitrary shapes and labels outliers, no k needed, struggles when densities differ. **Gaussian mixtures** — soft, elliptical clusters with probabilities.

PCA — dimensionality reduction

Finds the directions along which the data varies most and re-expresses it in a few of them. Uses: compressing many correlated features, visualising in 2-D, removing noise. Cost: the new components are mixtures of the original features — harder to interpret.

Anomaly detection

Finding the cases that don't look like the rest, without labels: z-scores, **isolation forest** (anomalies are easy to isolate with few random splits), distance to nearest neighbours, or a large reconstruction error. Use: new fraud patterns nobody has labelled yet.

30 SECONDS — "HOW DOES K-MEANS WORK?"

"It's a clustering algorithm for unlabelled data. I pick k; it places k centres, assigns each point to the nearest centre, moves each centre to the mean of its assigned points, and repeats until the assignments stop changing. It's fast and simple, but I have to choose k — with the elbow method or silhouette score, or by what the business can act on — it assumes roughly spherical clusters, and it's sensitive to feature scaling and to the starting points, so I standardise and run it several times."

D10 Preparing the data — the questions that come with the models

TOPIC	WHAT TO KNOW
Categorical features	One-hot encoding: one 0/1 column per category — needed because models do arithmetic and "country = 3" would imply an order. Too many categories (merchant ID) → target encoding, hashing, or learned embeddings. Tree libraries like CatBoost / LightGBM handle categories directly.
Scaling	Needed for anything based on distances or gradient descent on a penalised loss: k-NN, k-means, SVM, regularised linear models, neural nets. Not needed for trees.
Missing values	Impute (median, or a model) and add a "was missing" flag — missingness is often informative. Boosted-tree libraries handle missing values natively.
Outliers	Check whether they're errors or signal. Cap / winsorise, log-transform, use robust losses (MAE, Huber) or tree models.
Feature selection	Fewer features → less overfitting, faster, easier to explain and to maintain. Methods: filter (correlation with target), L1 regularisation, model-based importance, recursive elimination.
Curse of dimensionality	More features need exponentially more data to cover the space; distances become meaningless. Why "just add every column" backfires.
Hyperparameter tuning	Grid search: try every combination — thorough, explodes with the number of parameters. Random search: sample combinations — finds good regions far faster when only a few parameters matter. Bayesian optimisation: smarter sampling. Always with cross-validation.
Train ≠ production data	If the test distribution differs from training (new market, new season), performance drops. Detect by monitoring feature distributions; fix by retraining on recent data.

D11 Which model when — the comparison table

MODEL	INTERPRETABLE	NON-LINEAR	NEEDS SCALING	MAIN STRENGTH	MAIN WEAKNESS
Linear regression	✓✓	✗	if regularised	simple, explainable baseline	linear only; outliers
Logistic regression	✓✓	✗	if regularised	calibrated, explainable, regulator-friendly	misses interactions
k-NN	✓	✓	✓	no training, simple	slow to predict; high dimensions
Naïve Bayes	✓	✗	✗	fast text baseline	independence assumption
Decision tree	✓✓	✓	✗	readable rules	unstable, overfits
Random forest	~	✓	✗	robust, little tuning	large, less accurate than boosting
Gradient boosting	~	✓	✗	best accuracy on tabular data	tuning; can overfit
SVM	✗	✓ (kernels)	✓	high-dimensional, small data	doesn't scale; no probabilities

MODEL	INTERPRETABLE	NON-LINEAR	NEEDS SCALING	MAIN STRENGTH	MAIN WEAKNESS
Neural network	✗	✓✓	✓	images, text, sequences; learns features	data, compute, opacity
k-means	✓	—	✓	fast clustering	must choose k; round clusters

How to choose — the questions to ask, in order

1. **Is there a label?** No → clustering / anomaly detection. Yes → supervised.
2. **What kind of data?** Images, text, sequences → neural networks (probably a pre-trained one). Tabular → trees or linear models.
3. **Must every decision be explained** (credit, a regulator, a customer complaint)? → logistic regression or a shallow tree; or boosted trees with SHAP if the regulator accepts it.
4. **How much data?** Hundreds of rows → simple models. Hundreds of thousands and up → boosting. Billions of events → representation learning.
5. **Latency and cost at serving time?** A linear model answers in microseconds; a large network may not fit a 200 ms budget.
6. **Always:** start with a simple baseline, and only keep the complex model if it beats the baseline by enough to pay for its complexity.

30 SECONDS — "WHICH MODEL WOULD YOU USE TO PREDICT A BINARY OUTCOME, AND WHY?"

"I'd start with logistic regression as a baseline — fast, interpretable, well calibrated. If the data is tabular and the relationship is likely non-linear with interactions, I'd try gradient-boosted trees, which usually give the best accuracy there, with early stopping and cross-validation. I'd compare them on precision–recall at the operating threshold, not accuracy, since the outcome is usually rare. Then the choice depends on constraints: if every decision has to be explained to a regulator, the simpler model may win even if it scores lower; if the input is text or event sequences, I'd move to a neural approach."

EXERCISES D7–D11

1. Why does a neural network need non-linear activation functions? What happens with none?
2. You have 5,000 labelled rows of tabular customer data and need to predict churn. Neural network or gradient boosting? Give two reasons.
3. Revolut wants to read the fields from photos of ID documents. Which model family, and why not a random forest on the raw pixels?
4. You run k-means with $k = 4$ twice and get different clusters. Why? What do you do about it?
5. Which of these need feature scaling: k-NN, random forest, logistic regression with L2, k-means, XGBoost, a neural network?
6. A feature "merchant_id" has 2 million distinct values. Why is one-hot encoding a problem, and what are two alternatives?
7. Pick a model and justify in two sentences: (a) a credit decision that must be explainable to the customer; (b) fraud scoring on 50 million tabular transactions with a 100 ms budget; (c) routing support messages to the right team; (d) finding new, unlabelled fraud patterns; (e) "customers similar to this one" for a campaign.
8. Grid search over 6 hyperparameters with 5 values each: how many models? Why is random search usually the better use of the same budget?

D12 One-page cheat sheet

Linear regression	Weighted sum → number; minimises MSE. Interpretable baseline; linear only; outliers and collinearity hurt.
Logistic regression	Weighted sum → sigmoid → probability; log loss. Linear in log-odds; weights = odds ratios; calibrated; the scorecard standard.
k-NN	Vote of the k most similar cases. No training; slow prediction; scale features; fails in high dimensions. k = bias–variance dial.
Naive Bayes	Bayes + "features independent given the class". Fast text baseline; badly calibrated.
Decision tree	If/else splits chosen by Gini / entropy. Readable, non-linear, no scaling; unstable and overfits.
Random forest	Many deep trees on bootstrap samples + random features, averaged → lower variance. Robust, parallel, little tuning.
Gradient boosting	Shallow trees in sequence, each fitting the previous errors → lower bias. Best on tabular; tune learning rate, depth; early stopping.
SVM	Widest-margin boundary; kernels for curves. Good on small high-dimensional data; doesn't scale.
Neural network	Layers + activations, trained by backprop. Learns features; images, text, sequences; needs data and compute; opaque.
k-means	Assign to nearest centre, move centres, repeat. Choose k (elbow / silhouette); scale; round clusters only.
Scaling needed?	Yes: k-NN, k-means, SVM, regularised linear, neural nets. No: trees and their ensembles.
Default playbook	Baseline (logistic) → boosted trees → compare on the right metric → pick by constraints (explainability, latency, data).

A Answers — check only after you've written yours

D1–D2

1. You can't tell from the raw weights: they're in different units (per €k of income vs. per year of age). Standardise both features (or compare weight × the feature's standard deviation), then compare.
2. Perfect multicollinearity: the weights aren't uniquely defined — the fit fails or returns arbitrary, unstable values; predictions may be fine but interpretation is meaningless. Drop one of the three, or use ridge regularisation.
3. Odds are multiplied by $e^{0.7} \approx 2.0$ — each extra month roughly doubles the odds of default. $z = 0 \rightarrow p = \sigma(0) = 0.5$.
4. No — it only adds separate effects of age and utilisation. Add an interaction feature (young × high utilisation), or bin the features, or use a tree-based model, which finds interactions itself.
5. Explainability and regulation: every decision can be traced to a few weights and explained to the customer and the regulator; it's stable, well calibrated and easy to monitor. The accuracy gain from boosting may not be worth the governance cost. (Many banks use boosting as a challenger, or with SHAP explanations.)

D3–D6

1. Bias rises, variance falls. At $k = 1,000$ (all the data) it predicts the overall majority class / mean for everyone — the constant model.
2. Income: its numbers are thousands of times larger than age, so differences in income swamp the distance. Standardise first.
3. It assumes features are independent given the class. Transaction amount and merchant category; or "new device" and "new IP address" — strongly related, so naive Bayes double-counts the same evidence and becomes over-confident.
4. Parent: $1 - (0.8^2 + 0.2^2) = 0.32$. Left (60/0): 0. Right (20/20): $1 - (0.25 + 0.25) = 0.5$. Weighted: $0.6 \times 0 + 0.4 \times 0.5 = 0.20$. Impurity falls from 0.32 to 0.20 — a good split (it isolated 60 certain non-defaulters).
5. Random forest: it averages many trees trained on resampled data, so the instability (variance) of any single tree cancels out.
6. To decorrelate the trees. If one feature is dominant, every tree would split on it first and the trees would be near-copies; averaging near-copies doesn't reduce variance. Random subsets force diversity.
7. Overfitting: after round 300 the model is fitting noise in the training data. Use early stopping (keep the model at round ~ 300), and/or lower the learning rate, reduce depth, add subsampling / regularisation.
8. Importance measures how much the *model* relies on X to predict, not causation. X may be a proxy for something else, or share importance with correlated features. To claim an effect you need an experiment (Module 2).

D7–D11

1. A weighted sum of weighted sums is still a weighted sum — without non-linearities any depth collapses into a single linear model. Activations let the network represent curves and interactions.
2. Gradient boosting: on small/medium tabular data it usually matches or beats neural nets with far less tuning; 5,000 rows is too little for a network to learn useful representations; faster to train, easier to explain (SHAP).
3. A convolutional neural network (or a pre-trained vision / OCR model, fine-tuned). Pixels have spatial structure — a "3" is a "3" wherever it sits — which convolutions exploit; a forest treats every pixel as an unrelated column and needs to relearn the pattern at every position.
4. k-means starts from random centres and only finds a local optimum. Run it many times and keep the best (lowest within-cluster distance), use k-means++ initialisation, and check whether the clusters are stable — if not, the structure may not be real.
5. Need scaling: k-NN, logistic regression with L2, k-means, neural network. Don't: random forest, XGBoost.
6. Two million sparse columns: memory, slow training, and most categories have too few examples to learn from. Alternatives: target encoding (replace the ID by its historical outcome rate, computed without leakage), hashing, grouping rare values, or learned embeddings; or CatBoost's native handling.
7. (a) Logistic regression (scorecard) or a shallow tree — each decision explainable. (b) Gradient-boosted trees — best accuracy on tabular data and fast at inference; logistic regression if the latency budget were tighter. (c) Text classification: a fine-tuned language model / LLM, with naive Bayes or logistic regression on bag-of-words as the baseline. (d) Unsupervised anomaly detection (isolation forest, distance in embedding space) feeding human review. (e) Nearest neighbours in an embedding space (e.g. customer embeddings), or clustering.
8. $5^6 = 15,625$ models ($\times$ the number of CV folds). Usually only two or three hyperparameters really matter; a grid wastes most runs varying the unimportant ones, while random search tries a new value of the important ones on every run.